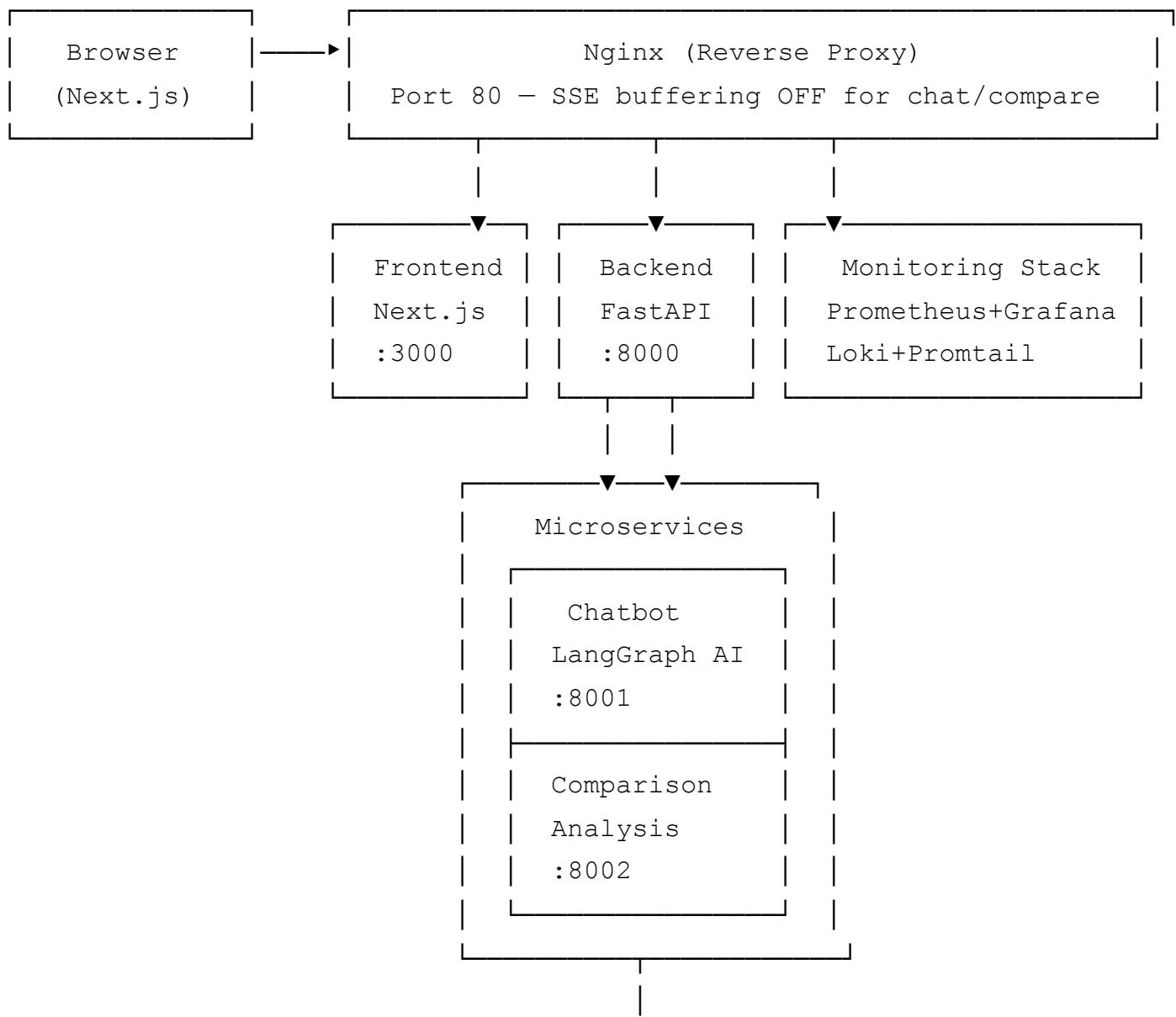


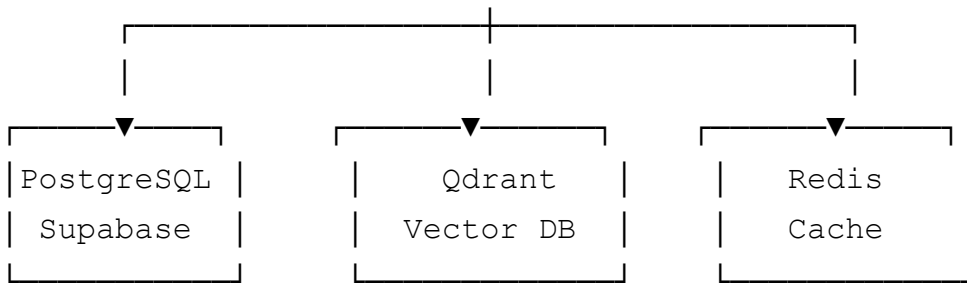
Deals – Technical Report

Project: GenZCars-Team – AI-Powered Car Marketplace for the Egyptian Market
Stack: Next.js 16 · FastAPI · LangGraph · Qdrant · PostgreSQL · Docker
Repository: <https://github.com/nhahub/NHA-4-140.git>

1. System Architecture Overview

Deals is a **3-microservice** platform with a modern Next.js frontend, fully containerized via Docker Compose. The system enables car buyers and sellers to interact through a conversational AI assistant, perform semantic searches, compare vehicles, and receive AI-driven market price analysis – all in **Arabic and English**.





2. Services Breakdown

2.1 Frontend (Next.js 16 – Port 3000)

Aspect	Detail
Framework	Next.js 16 (React 18) with App Router
Styling	Tailwind CSS 3 + Framer Motion
State	Zustand (auth, chat, compare stores)
Forms	React Hook Form + Zod validation
i18n	next-intl (English + Arabic RTL, Cairo font)
Charts	Recharts (radar, bar charts for comparison)
Build	<code>output: 'standalone'</code> for Docker

Pages:

- `/` – Hero section + featured cars + AI showcase
- `/ads` – Ad grid with filters (brand, model, price, year, etc.)
- `/ads/[id]` – Ad detail with image gallery, specs, AI advisor
- `/ads/new` – Create car listing
- `/search` – Semantic search results
- `/compare` – AI-powered comparison report
- `/chat` – Full-page conversational AI
- `/profile` , `/profile/settings` – User management
- `/login` , `/register` – Authentication forms
- `/about` – About page

Key Components:

- `ChatBubble` + `ChatDrawer` – Floating chat interface accessible site-wide
- `CompareTray` – Persistent bottom tray for car comparison

- `HeroSection` + `AIShowcase` — Landing page marketing
- `AdCard` , `AdGrid` , `AdFilters` , `AdForm` — Full ad management UI
- `ScoreRadar` , `HeadToHeadTable` , `VerdictCard` — Comparison visualization

2.2 Backend (FastAPI – Port 8000)

API Gateway handling auth, CRUD, search, and proxying to microservices.

Route Group	Endpoints	Purpose
<code>/api/v1/auth</code>	<code>register</code> , <code>login</code> , <code>refresh</code> , <code>logout</code>	JWT with httpOnly cookies
<code>/api/v1/users</code>	<code>me</code> , <code>me/avatar</code> , <code>{id}/ads</code>	Profile management
<code>/api/v1/ads</code>	CRUD + images + brand-counts	Car listing management
<code>/api/v1/search</code>	<code>?q=</code>	Hybrid semantic + BM25 search
<code>/api/v1/favorites</code>	<code>{ad_id}</code>	Save/favorite listings
<code>/api/v1/chat</code>	<code>session</code> , <code>message</code> (SSE), <code>stt</code> , <code>tts</code> , <code>history</code>	Chat + voice proxy
<code>/api/v1/compare</code>	SSE proxy → Comparison service	AI comparison
<code>/api/v1/compare-ai</code>	SSE (local)	Local AI comparison with vision
<code>/api/v1/price-analysis</code>	Proxy → Comparison service	Market price estimation
<code>/api/v1/analytics</code>	<code>stats</code>	Site statistics
<code>/api/v1/site-assets</code>	Logo/wallpaper URLs	Dynamic branding

AI Comparison Engine (in Backend):

- `structured_analyzer.py` — LLM analysis of car specs
- `comparison_agent.py` — Head-to-head LLM comparison
- `vision_analyzer.py` — OpenRouter vision model for photo analysis
- `hallucination_guard.py` — 5 validation checks on output
- `service.py` — Orchestrator: load → analyze → compare → validate → format

2.3 Chatbot (LangGraph – Port 8001)

10-node conversational state machine with MCP tool architecture.

Graph Flow:

preference_extractor → router → (catalogue | search | recommend | advisor | seller | guide | general) → responder

Node	Function
preference_extractor	Two-pass preference extraction from user input
router	LLM intent classification (find, compare, sell, guide, general)
catalogue_node	Catalogue availability check
search_node	Query builder + Qdrant hybrid search
recommendation_node	Alternative car recommendations
advisor_node	"Is this a good deal?" analysis
seller_node	Seller guidance
guide_node	Website usage FAQ
general_node	Open-domain car knowledge
responder_node	SSE streaming + DB persistence

MCP Tool Servers (10 tools in-process):

Server	Tools
car_search_server	search_cars , get_car_details , find_similar_cars , check_catalogue , get_car_images
analysis_server	analyze_market_price , web_search , get_website_guide , expand_brand_origins , expand_colloquial_terms

LLM Fallback Chain:

Tier	Provider	Model
1(Fast)	Groq	openai/gpt-oss-120b (temp 0.0)
2(Powerful)	Groq	qwen/qwen3.6-27b (temp 0.3)
3(Fallback)	OpenRouter	nvidia/nemotron-3-nano (temp 0.3)

Voice:

- **STT:** Groq Whisper(whisper-large-v3-turbo)with multi-key fallback
- **TTS:** edge-tts(ar-EG-SalmaNeural , en-US-JennyNeural)

2.4 Comparison Analysis (FastAPI – Port 8002)

Standalone microservice for AI comparison and price analysis with web research.

Endpoint	Method	Description
<code>/compare</code>	POST (SSE)	Compare 2-3 cars by ad_ids
<code>/price-analysis</code>	POST	Market price estimation

Pipeline:

1. Load ads from PostgreSQL
2. Research via Tavily (3 queries per car: reliability, price, reputation)
3. Analyze each car individually (LLM + research context)
4. Head-to-head comparison (LLM)
5. Build structured report with scoring, verdict
6. Cache result in Redis + SSE stream to client

Research Sources:

- **DuckDuckGo** – 7 parallel queries per price analysis
- **Tavily** – 3 research queries per car

3. Database Schema (PostgreSQL)

Tables auto-created on startup by `Backend/app/db/client.py` :

Table	Purpose
<code>users</code>	User accounts with password hashing
<code>ads</code>	Car listings: brand, model, year, price, km_driven, fuel, transmission, body_type, condition, city, description, images, etc.
<code>ad_images</code>	Image URLs per ad
<code>favorites</code>	User-ad favorites (user_id, ad_id)
<code>ad_views</code>	View tracking with IP deduplication
<code>chat_sessions</code>	Per-session tracking
<code>chat_messages</code>	Full conversation history

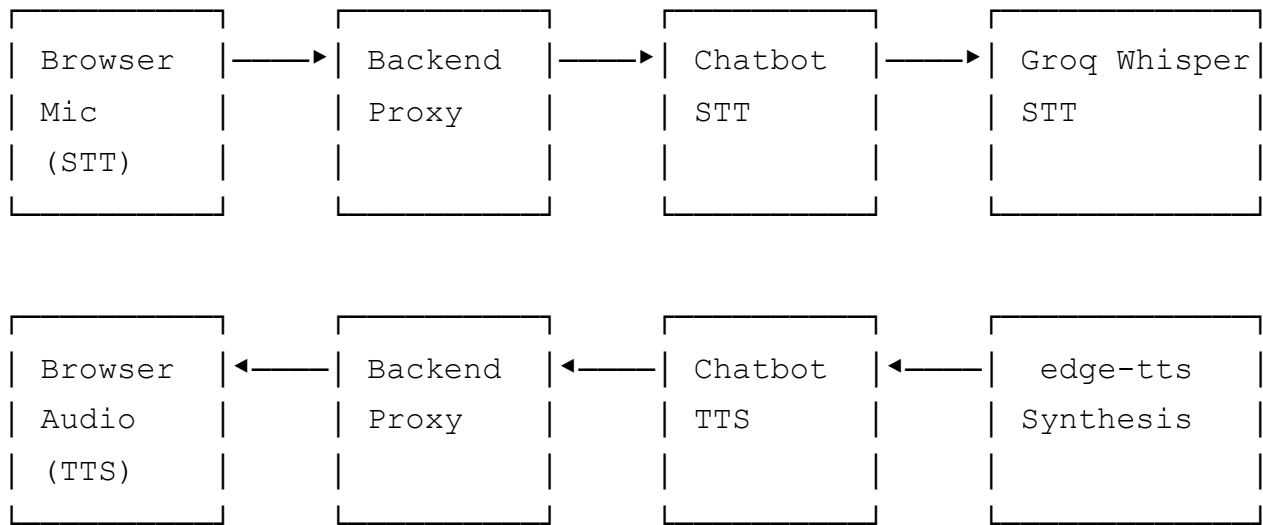
Table	Purpose
user preferences	Accumulated preference state for chat

4. Search Architecture

Component	Detail
Embedding Model	sentence-transformers/all-MiniLM-L6-v2 (384-dim)
Vector DB	Qdrant – Collection cars_ads , Cosine distance
Search Mode	Hybrid (dense vector + BM25 keyword)
Payload Indexes	11 fields: brand, model, city, fuel, transmission, body_type (keyword), price (float), year, km_driven (int), is_active (bool)
Embedding Text	brand + " " + model + " " + year + " " + body type + " " + fuel type + " " + transmission + " " + condition + " " + city + " " + cc range + " " + description + " " + special_conditions

Search is handled in Backend/app/routers/search.py (API endpoint) and Chatbot/app/core/qdrant_client.py (agent tool).

5. Voice Architecture



- **Client:** MediaRecorder capture → useVoice hook → STT endpoint
- **Server:** Groq Whisper (multi-key fallback) → edge-tts (Arabic/English neural voices)

6. Infrastructure

Docker Compose – 9 Containers

Service	Container	Port	Base
Frontend	<code>cars_frontend</code>	3000	Node.js (Next.js)
Nginx	<code>cars_nginx</code>	80	nginx:alpine
Backend	<code>cars_backend</code>	8000	Python (FastAPI)
Chatbot	<code>cars_chatbot</code>	8001	Python (FastAPI + LangGraph)
Comparison	<code>cars_comparison</code>	8002	Python (FastAPI)
Redis	<code>cars_redis</code>	6379	redis:alpine
Prometheus	<code>cars_prometheus</code>	9090	prom/prometheus
Grafana	<code>cars_grafana</code>	3001	grafana/grafana
Loki + Promtail	<code>cars_loki</code>	3100	grafana/loki

Monitoring

- **Prometheus** scrapes `/metrics` on all 3 Python services (every 15s)
- **Grafana** pre-provisioned with Prometheus + Loki datasources + FastAPI dashboard
- **Loki** collects Docker logs via Promtail

Nginx Configuration

- SSE endpoints (`/api/v1/chat/` , `/api/v1/compare`) – 3600s/600s timeout, `proxy_buffering off`
- Regular API – 120s timeout
- WebSocket support for HMR (`/_next/webpack-hmr`)
- Frontend served at `/` , API at `/api/`

7. Technology Stack Summary

Layer	Technology
-------	------------

Layer	Technology
Frontend	Next.js 16, React 18, Tailwind CSS 3, Framer Motion, Zustand, Recharts, next-intl
API Gateway	FastAPI (Python 3.11+), asyncpg, Pydantic v2
AI/LLM	LangGraph, LangChain, Groq (primary), OpenRouter (fallback)
Vector DB	Qdrant (all-MiniLM-L6-v2, 384-dim, cosine)
Database	PostgreSQL (via asyncpg), Supabase (file storage)
Cache	Redis (async hiredis)
Voice	Groq Whisper STT, edge-tts TTS
Search	Hybrid: dense vector + BM25 (Qdrant)
Monitoring	Prometheus, Grafana, Loki
Containerization	Docker Compose (9 services), Nginx reverse proxy
Auth	JWT (access + refresh tokens), httpOnly cookies, bcrypt
Web Research	Tavily API, DuckDuckGo Search

8. Key Design Decisions

- Three-tier LLM fallback** ensures reliability even when primary providers are rate-limited or down.
- In-process MCP tools** avoid network overhead of external MCP servers while maintaining clean separation of concerns.
- SSE streaming** for all AI responses enables real-time UX with progressive loading indicators.
- Hybrid search** combines semantic understanding with keyword precision for Egyptian car market terminology.
- Hallucination guard** with 5 validation checks ensures AI-generated comparison data is grounded in actual car specs.
- Multi-key fallback** for Groq API keys distributes rate limits across multiple accounts.
- Arabic/English bilingual support** through next-intl with RTL layout and Arabic neural TTS voice.

9. Development Setup

```
# Prerequisites: Docker, Docker Compose

# Clone and navigate
cd genzcars-team
```

```

# Start all services
cd infra
make setup
make build
make up

# Or manually:
docker-compose --env-file .env.example up --build

# Access
# Frontend:  http://localhost
# API:       http://localhost/api/v1
# Grafana:   http://localhost:3001
# Prometheus: http://localhost:9090

```

10. Project Structure

```

genzcars-team/
├── Backend/                                # API Gateway (FastAPI, port 8000)
│   ├── app/
│   │   ├── core/                          # LLM, cache, security, middleware, metrics
│   │   ├── db/                            # Database queries (PostgreSQL)
│   │   │   └── queries/                   # SQL modules: ads, users, favorites, views
│   │   ├── routers/                      # API route handlers
│   │   ├── schemas/                     # Pydantic models
│   │   └── services/                     # Business logic + comparison engine
│   └── Dockerfile
├── Chatbot/                               # Conversational AI (LangGraph, port 8001)
│   ├── app/
│   │   ├── agents/                       # LangGraph agent nodes
│   │   ├── core/                         # LLM, MCP, Qdrant, guardrails, cache, metrics
│   │   │   └── mcp/servers/              # 10 MCP tools (2 servers)
│   │   ├── data/                         # Brand origins, colloquial terms, constants
│   │   ├── db/                           # Chat history queries
│   │   ├── graph/                       # State machine: builder, router, state, nodes
│   │   ├── routers/                     # API endpoint handlers
│   │   ├── schemas/                     # Pydantic models
│   │   └── voice/                        # STT + TTS endpoints
│   └── Dockerfile
└── Comparison_Analysis/                   # AI Comparison + Price Analysis (port 8002)

```

			app/				
				core/			
				# LLM, cache, web search, metrics			
				db/			
				# Ad fetching queries			
				pipeline/			
				# Analyzer, comparator, report builder, runner			
				routers/			
				# API endpoint handlers			
					schemas/		
				# Pydantic models			
					Dockerfile		
				Frontend/			
				# Next.js 16 Web Application			
					app/		
				# App Router pages			
					(auth)/		
				# Login, register			
					ads/		
				# Ad detail, creation			
					chat/		
				# Full chat page			
					compare/		
				# Comparison report			
					profile/		
				# User profile, settings			
						search/	
				# Search results			
					components/		
				# React components			
						ads/	
				# Ad cards, forms, filters, gallery			
						chat/	
				# Chat UI (bubble, drawer, input, messages)			
						compare/	
				# Comparison visualization			
						home/	
				# Landing page sections			
						layout/	
				# Navbar, footer, mobile nav, page transitions			
							ui/
				# Reusable UI primitives			
					hooks/		
				# Custom React hooks			
					lib/		
				# API client, utils, SSE parser			
					store/		
				# Zustand state stores			
					messages/		
				# i18n translations (en, ar)			
						types/	
				# TypeScript type definitions			
					infra/		
				# Infrastructure			
						docker-compose.yml	
				# 9-service deployment			
						nginx/	
				# Reverse proxy configuration			
						prometheus/	
				# Metrics scraping config			
						grafana/	
				# Dashboards + datasources			
							loki/
				# Log aggregation			
					docs/		
				# Documentation, architecture, presentations			
						logos/	
				# 24 technology SVG icons			
						voice assistant/	
				# Voice architecture reference			